EXPERIMENT NO: 5
CLUSTERING WITH K-MEANS AND DIMENSIONALITY REDUCTION WITH PCA

AIM

To demonstrate the application of Unsupervised Learning models, specifically K-Means clustering for grouping data points and Principal Component Analysis (PCA) for dimensionality reduction and visualization, using a suitable dataset.

PROCEDURE

1. Import the required libraries: NumPy, Pandas, Matplotlib, Seaborn, and the necessary scikit-learn modules.
2. Generate a two-dimensional dataset containing 300 samples and three clusters using make_blobs.
3. Apply the Elbow Method by fitting K-Means for K values from 1 to 10 and plotting the WCSS values.
4. Fit K-Means with K = 3, assign cluster labels, plot the clusters and centroids, and calculate the silhouette score.
5. Generate a four-dimensional dataset containing 500 samples and four clusters.
6. Standardize the dataset using StandardScaler, then apply PCA to reduce the four features to two principal components.
7. Print the original dataset head, reduced dataset head, explained variance ratio, and total explained variance.
8. Plot the PCA-reduced data, apply K-Means with K = 4 to the reduced data, visualize the clusters and centroids, and calculate the silhouette score.

CODE

```
# ==============================
# EXPERIMENT 5 – K-Means & PCA
# ==============================

# Import necessary libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import make_blobs
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score

# --- Part 1: K-Means Clustering ---
print("--- Part 1: K-Means Clustering ---")

# 1. Generate dataset
X, y = make_blobs(n_samples=300, centers=3, cluster_std=0.60, random_state=42)
df_kmeans = pd.DataFrame(X, columns=['Feature_1', 'Feature_2'])
print("\nOriginal K-Means Dataset Head:")
print(df_kmeans.head())

# 2. Elbow Method
wcss = []
for i in range(1, 11):
    kmeans = KMeans(
        n_clusters=i,
```

```
        init='k-means++',
        max_iter=300,
        n_init=10,
        random_state=42
    )
    kmeans.fit(X)
    wcss.append(kmeans.inertia_)

plt.figure(figsize=(10, 6))
plt.plot(range(1, 11), wcss, marker='o', linestyle='--')
plt.title('Elbow Method for Optimal K (K-Means)')
plt.xlabel('Number of Clusters (K)')
plt.ylabel('WCSS')
plt.grid(True)
plt.tight_layout()
plt.savefig("exp05_output_1_elbow.png", dpi=150)
plt.close()

# 3. Apply K-Means with chosen K
optimal_k = 3
kmeans = KMeans(
    n_clusters=optimal_k,
    init='k-means++',
    max_iter=300,
    n_init=10,
    random_state=42
)
clusters = kmeans.fit_predict(X)
df_kmeans['Cluster'] = clusters

# 4. Visualize K-Means clusters
plt.figure(figsize=(10, 8))
sns.scatterplot(
    x='Feature_1', y='Feature_2', hue='Cluster',
    data=df_kmeans, palette='viridis', s=100, alpha=0.8
)
plt.scatter(
    kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1],
    s=300, c='red', marker='X', label='Centroids'
)
plt.title(f'K-Means Clustering with K={optimal_k}')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.savefig("exp05_output_2_kmeans.png", dpi=150)
plt.close()

# 5. Silhouette Score
silhouette_avg = silhouette_score(X, clusters)
print(f"\nSilhouette Score for K-Means (K={optimal_k}): {silhouette_avg:.3f}")

# --- Part 2: Dimensionality Reduction with PCA ---
print("\n--- Part 2: Dimensionality Reduction with PCA ---")

# 1. Generate 4D dataset
X_pca, y_pca = make_blobs(
    n_samples=500, n_features=4, centers=4,
    cluster_std=1.0, random_state=25
)
df_pca_original = pd.DataFrame(
    X_pca, columns=[f'Feature_{i+1}' for i in range(X_pca.shape[1])]
)
df_pca_original['True_Cluster'] = y_pca
print("\nOriginal PCA Dataset Head:")
print(df_pca_original.head())
print(f"Original PCA Dataset Shape: {df_pca_original.shape}")

# 2. Standardize
scaler = StandardScaler()
```

```
X_pca_scaled = scaler.fit_transform(X_pca)

# 3. PCA (4D -> 2D)
pca = PCA(n_components=2)
principal_components = pca.fit_transform(X_pca_scaled)
df_principal_components = pd.DataFrame(
    principal_components,
    columns=['Principal_Component_1', 'Principal_Component_2']
)
df_principal_components['True_Cluster'] = y_pca
explained_variance = pca.explained_variance_ratio_
print("\nPrincipal Components Head:")
print(df_principal_components.head())
print(f"\nExplained Variance Ratio: {explained_variance}")
print(f"Total Explained Variance by 2 PCs: {explained_variance.sum():.3f}")

# 4. Visualize PCA result
plt.figure(figsize=(10, 8))
sns.scatterplot(
    x='Principal_Component_1', y='Principal_Component_2',
    hue='True_Cluster', data=df_principal_components,
    palette='Paired', s=100, alpha=0.8
)
plt.title('PCA - Dimensionality Reduction to 2 Components')
plt.xlabel(f'PC1 ({explained_variance[0]*100:.2f}%)')
plt.ylabel(f'PC2 ({explained_variance[1]*100:.2f}%)')
plt.grid(True)
plt.tight_layout()
plt.savefig("exp05_output_3_pca.png", dpi=150)
plt.close()

# 5. K-Means on PCA-reduced data
kmeans_pca = KMeans(
    n_clusters=4, init='k-means++', max_iter=300,
    n_init=10, random_state=42
)
clusters_pca = kmeans_pca.fit_predict(principal_components)
df_principal_components['KMeans_Cluster_on_PCA'] = clusters_pca

plt.figure(figsize=(10, 8))
sns.scatterplot(
    x='Principal_Component_1', y='Principal_Component_2',
    hue='KMeans_Cluster_on_PCA', data=df_principal_components,
    palette='viridis', s=100, alpha=0.8
)
plt.scatter(
    kmeans_pca.cluster_centers_[:, 0], kmeans_pca.cluster_centers_[:, 1],
    s=300, c='red', marker='X', label='Centroids'
)
plt.title('K-Means Clustering on PCA-Reduced Data')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.savefig("exp05_output_4_kmeans_pca.png", dpi=150)
plt.close()

# 6. Silhouette Score for PCA-reduced KMeans
silhouette_avg_pca = silhouette_score(principal_components, clusters_pca)
print(
    f"\nSilhouette Score for K-Means on PCA-Reduced Data (K=4): "
    f"{silhouette_avg_pca:.3f}"
)
```

OUTPUT

Actual console output produced by running the code:

```
--- Part 1: K-Means Clustering ---

Original K-Means Dataset Head:
   Feature_1  Feature_2
0  -7.155244  -7.390016
1  -7.395875  -7.110843
2  -2.015671   8.281780
3   4.509270   2.632436
4  -8.102502  -7.484961

Silhouette Score for K-Means (K=3): 0.908

--- Part 2: Dimensionality Reduction with PCA ---

Original PCA Dataset Head:
   Feature_1  Feature_2  Feature_3  Feature_4  True_Cluster
0  -0.638667   1.110057  -6.400722  -0.204990             3
1  -2.951556  -7.657445   3.844794   0.903589             1
2  -0.253177   2.125103  -7.869801   0.559678             3
3  -2.151209   3.401400  -5.734930   0.965230             3
4  -2.347519  -7.230467   3.478891  -0.443440             1
Original PCA Dataset Shape: (500, 5)

Principal Components Head:
   Principal_Component_1  Principal_Component_2  True_Cluster
0               0.455305               1.623917             3
1              -2.705622               0.375012             1
2               0.810234               1.966926             3
3               0.427139               2.149626             3
4              -2.407508               0.099250             1

Explained Variance Ratio: [0.57208431 0.33622342]
Total Explained Variance by 2 PCs: 0.908

Silhouette Score for K-Means on PCA-Reduced Data (K=4): 0.776
```

Graphical output:

Figure 1: Elbow Method for Optimal K (K-Means)

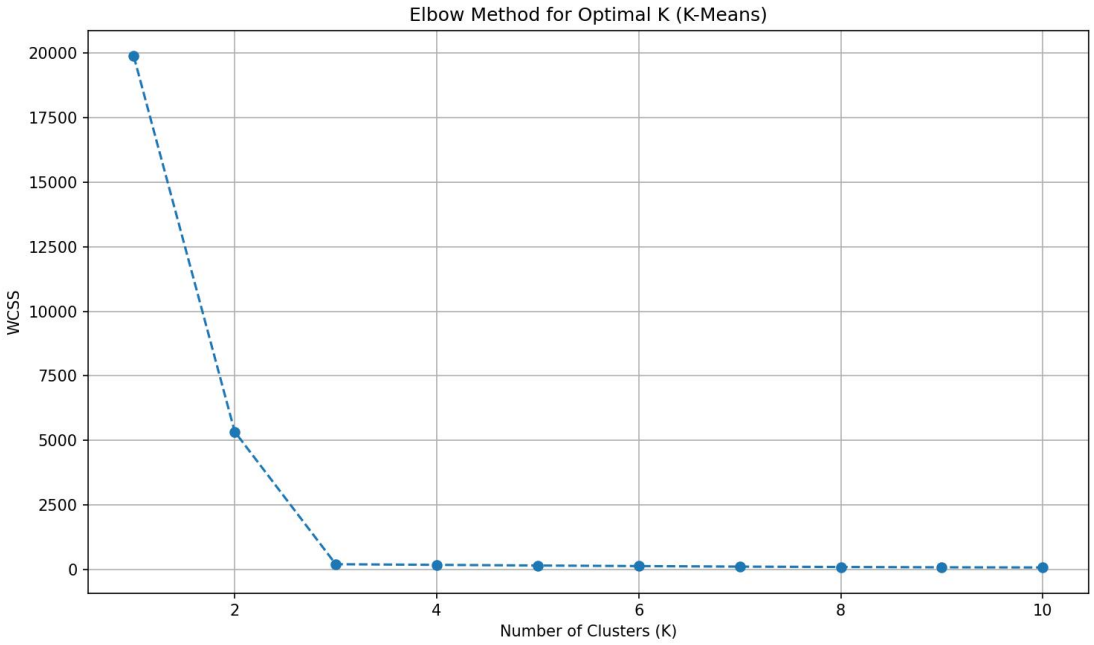


Figure 2: K-Means Clustering with K=3

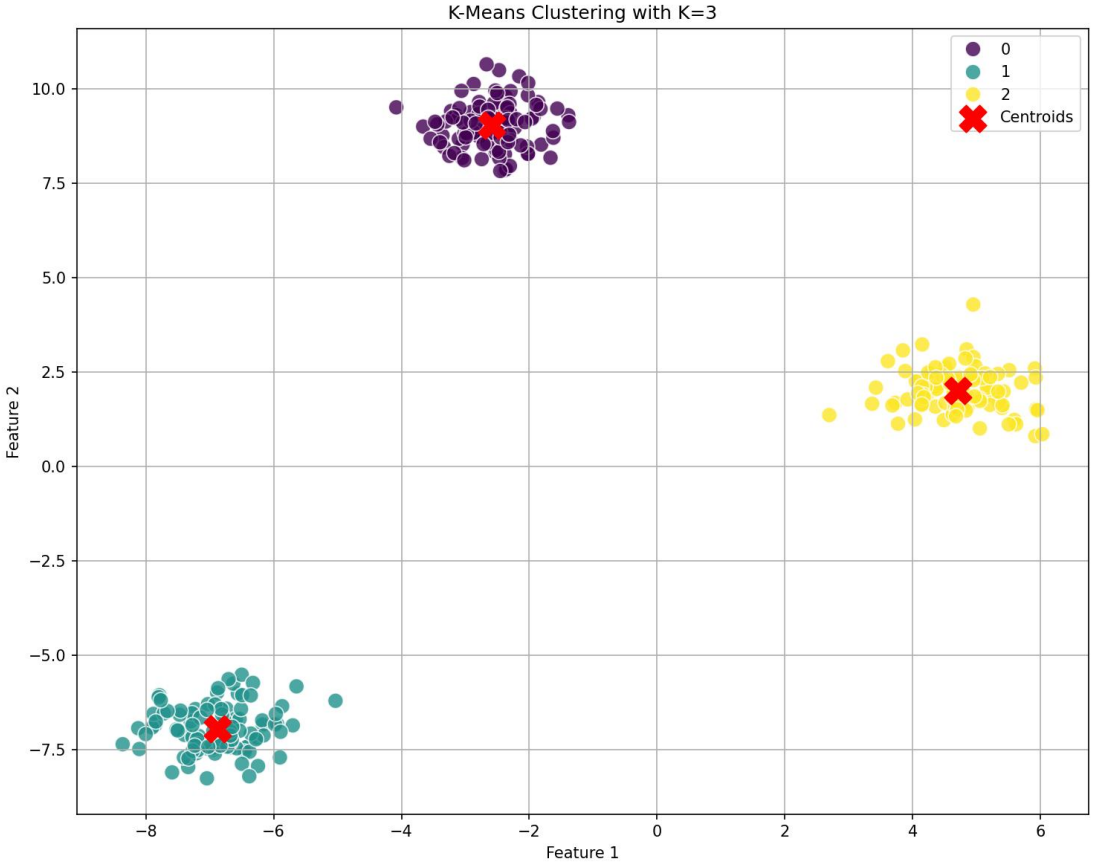


Figure 3: PCA – Dimensionality Reduction to 2 Components

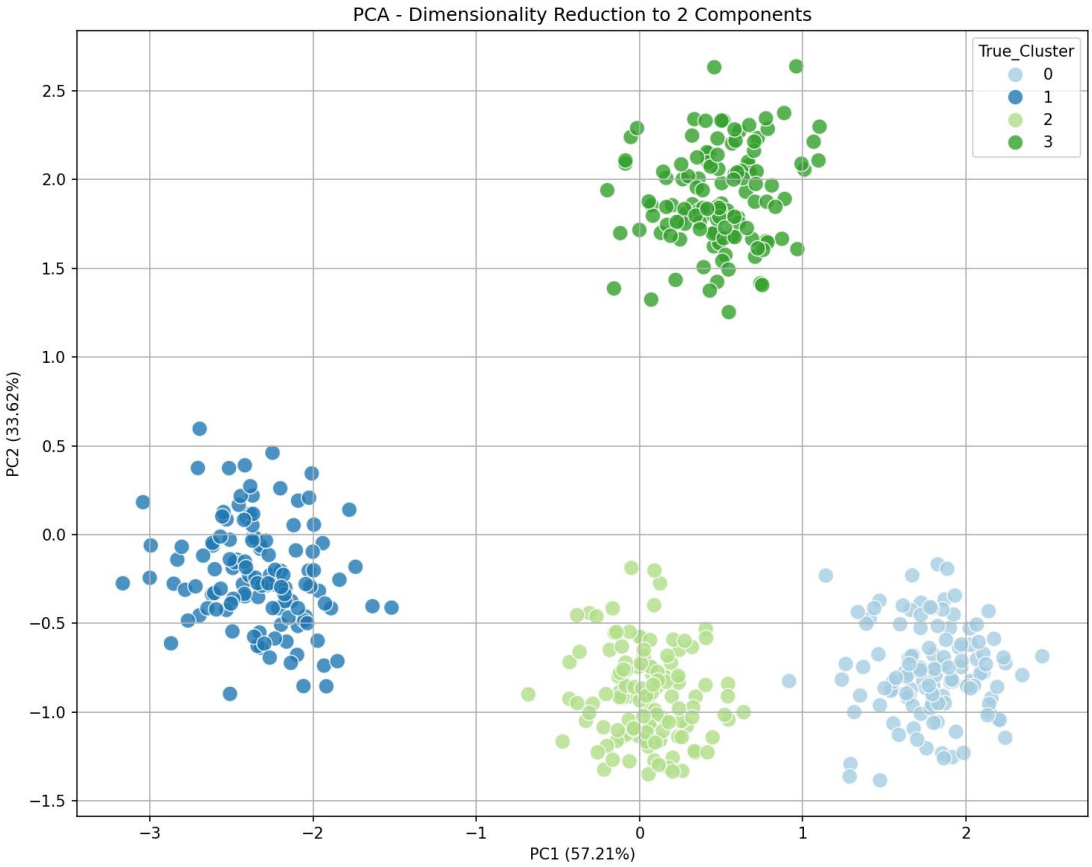
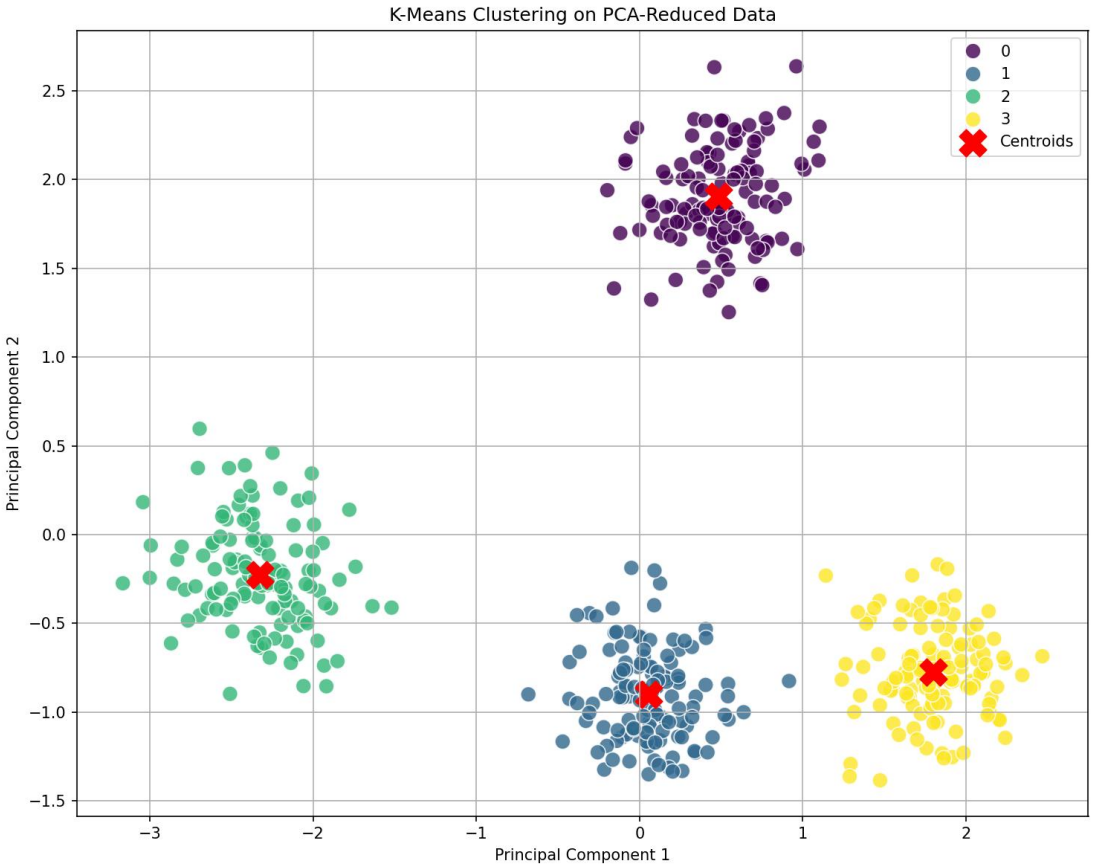


Figure 4: K-Means Clustering on PCA-Reduced Data



RESULT

The K-Means clustering and Principal Component Analysis (PCA) techniques were successfully implemented on the given dataset. K-Means grouped the data points into distinct clusters based on feature similarity. PCA reduced the four-dimensional dataset to two principal components while retaining the variance indicated by the printed explained-variance ratio. K-Means was then applied to the PCA-reduced data to identify clusters, and silhouette scores were calculated to evaluate the clustering results.